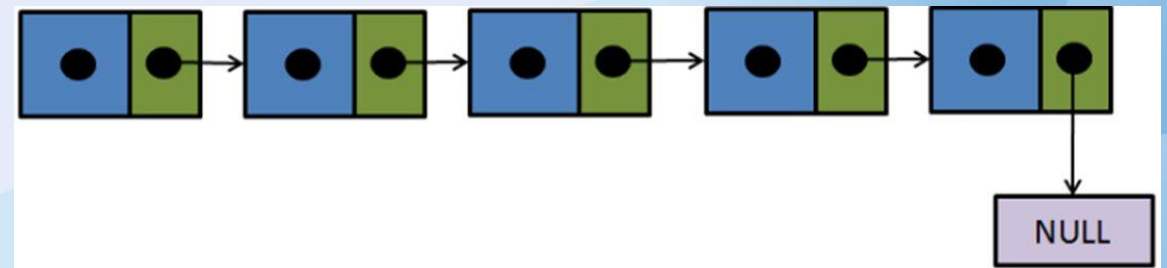


Estrutura de Dados I

Prof.: Michele Nasu Tomiyama Bucci

Lista Linear

- Uma lista linear é um conjunto de n elementos (de informações):
 - **$x_1, x_2, \dots, x_n$,**
- cuja propriedade estrutural envolve as posições relativas de seus elementos.
- Supondo $n > 0$, temos:
 - **x_1** é o primeiro elemento
 - De x_2 até **x_{n-1}** são os elementos centrais
 - **x_n** é o último elemento.



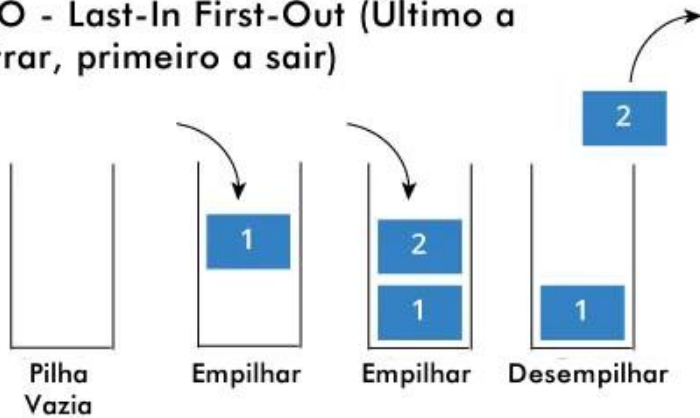
- Algumas operações que podemos querer realizar sobre listas lineares:
 - Ter acesso a **x_k** , k qualquer, a fim de examinar ou alterar o conteúdo de seus campos
 - Inserir um elemento novo antes ou depois de **x_k**
 - Remover **x_k**
 - Colocar todos os elementos da lista em ordem.
 - Combinar 2 ou mais listas lineares em uma só
 - Quebrar uma lista linear em duas ou mais
 - Copiar uma lista linear em um outro espaço

- A maneira de implementar listas lineares depende da classe de operações mais frequentes.
- Não existe, em geral, uma única implementação para a qual todas as operações são eficientes.
- Por exemplo, **não existe uma implementação para atender às seguintes duas operações de maneira eficiente:**
 - **1** ter acesso fácil ao x_k , para k qualquer
 - **2** inserir ou remover elementos em qualquer posição da lista linear
- A operação **1** fica eficiente se a lista é implementada em um vetor (array) em alocação sequencial na memória.
- Já para a operação **2** é mais adequada a alocação encadeada ou ligada, com o uso de apontadores.

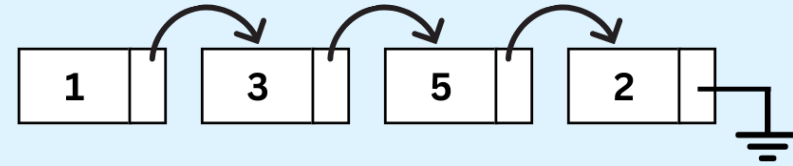
- Tipos de Listas

- Fila
- Pilha
- Deque
- Lista

LIFO - Last-In First-Out (Último a entrar, primeiro a sair)



LISTA ENCADEADA



FIFO - First-In First-Out (Primeiro a entrar, primeiro a sair)



Deque



Pilhas (Stack)

- Listas lineares em que inserções, remoções e acessos a elementos ocorrem no primeiro ou no último elemento são muito frequentemente encontradas.
- Tais listas lineares recebem nomes especiais como pilha e fila.
- A **Pilha** é uma lista linear em que todas as inserções e remoções são feitas numa mesma extremidade da lista linear.
- Esta extremidade se denomina **topo** (em inglês “top”) ou lado aberto da pilha.



- As operações definidas para uma pilha incluem:
 - Verificar se a pilha está **vazia**
 - **Inserir** um elemento na pilha (empilhar ou “push”)
 - **Remover** um elemento da pilha (desempilhar ou “pop”)

LIFO - Last-In First-Out (Último a entrar, primeiro a sair)



- Novos itens podem ser colocados no topo da pilha:
 - neste caso o topo será deslocado para cima, de modo a corresponder ao novo primeiro elementos.
- Ou os itens que estiverem no topo da pilha poderão ser removidos:
 - neste caso o topo será deslocado para baixo, de modo a corresponder ao novo primeiro elemento.



- Como o último elemento que entrou na pilha será o primeiro a sair da pilha, a pilha é conhecida como uma estrutura do tipo **LIFO** (“**Last In First Out**”).
- Na vida real: pilhas de pratos numa cafeteria (acréscimos e retiradas de pratos sempre feitos num mesmo lado da pilha- lado de cima)
- Na execução de uma programa: uma pilha pode ser usada na chamada de procedimentos, para armazenar o endereço de retorno (e os parâmetros reais). A medida que procedimentos chamam outros procedimentos, mais e mais endereços de retorno devem ser empilhados. Estes são desempilhados à medida que os procedimentos chegam ao seu fim.

TAD NÓ

- Como vamos trabalhar com a implementação dinâmica das estruturas, vamos precisar da classe NO.

```
1 public class No {
2     private Object valor;
3     private No prox;
4
5     public No(Object valor, No prox) {
6         this.valor = valor;
7         this.prox = prox;
8     }
9     public No(){
10
11     }
12
13     public Object getValor() {
14         return valor;
15     }
16
17     public void setValor(Object valor) {
18         this.valor = valor;
19     }
20
21     public No getProx() {
22         return prox;
23     }
24
25     public void setProx(No prox) {
26         this.prox = prox;
27     }
28 }
```

TAD Pilha

- Como a estrutura faz uso de alocação dinâmica de memória, a estrutura da pilha só precisa saber quem é o topo da pilha.

```
1 public class Pilha {  
2     private No topo;  
3 }
```

Pilha topo = ???

- A estrutura não precisa saber exatamente onde todas as informações da pilha está armazenada, ela precisa saber somente onde está o topo da pilha.

```

1  public class Pilha {
2      private No topo;

3
4      public Pilha(No topo) {
5          this.topo = null;
6      }
7      public Pilha() {
8      }

9
10     public No getTopo() {
11         return topo;
12     }

13
14     public void setTopo(No topo) {
15         this.topo = topo;
16     }
17 }

```

Construtora: cria uma pilha vazia. Se o topo esta apontando para **null**, a pilha está vazia, não armazena nenhuma informação.

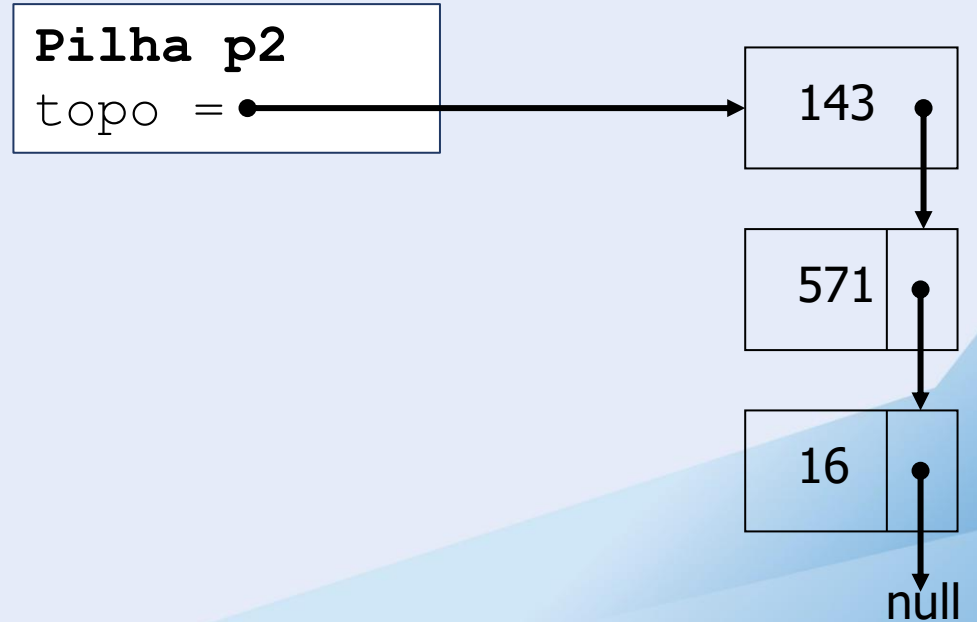
Retorna o Nó que está no topo da pilha.

Atualiza o topo da pilha. Sempre que um Nó é inserido ou removido da estrutura, o topo deve ser atualizado.

- A operação **vazia()** verifica se a pilha está vazia, ou seja, não armazena nenhuma informação.

```
15     public boolean vazia(){  
16         return this.topo == null;  
17     }
```

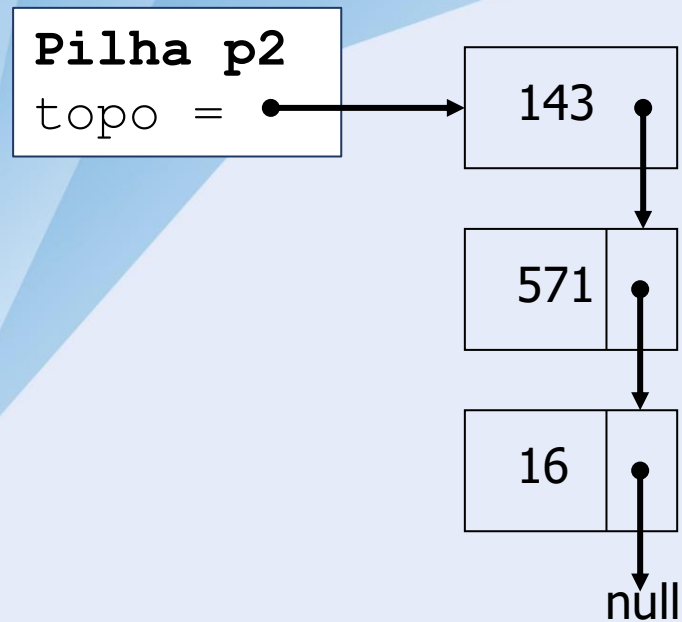
Pilha p1
topo = null



- A operação **inserir(Object x)** insere um novo nó na pilha. Este novo nó será o novo topo da pilha.

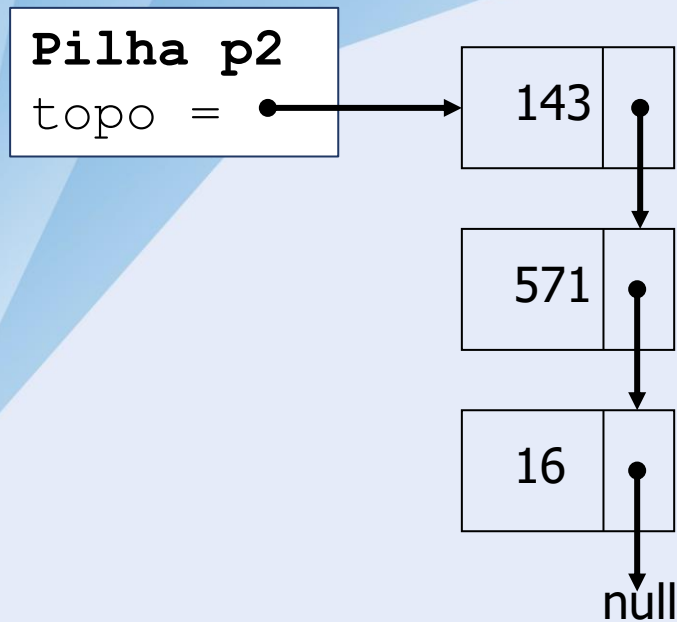
```
18     public void inserir(Object x){  
19         No novo = new No();  
20         novo.setValor(x);  
21         novo.setProx(this.topo);  
22         this.topo = novo;  
23     }
```

- A inserção em uma pilha requer a criação de um novo nó, atribuir as informações de **valor** e **prox**, e por fim, fazer o **topo** da pilha apontar para o novo nó.

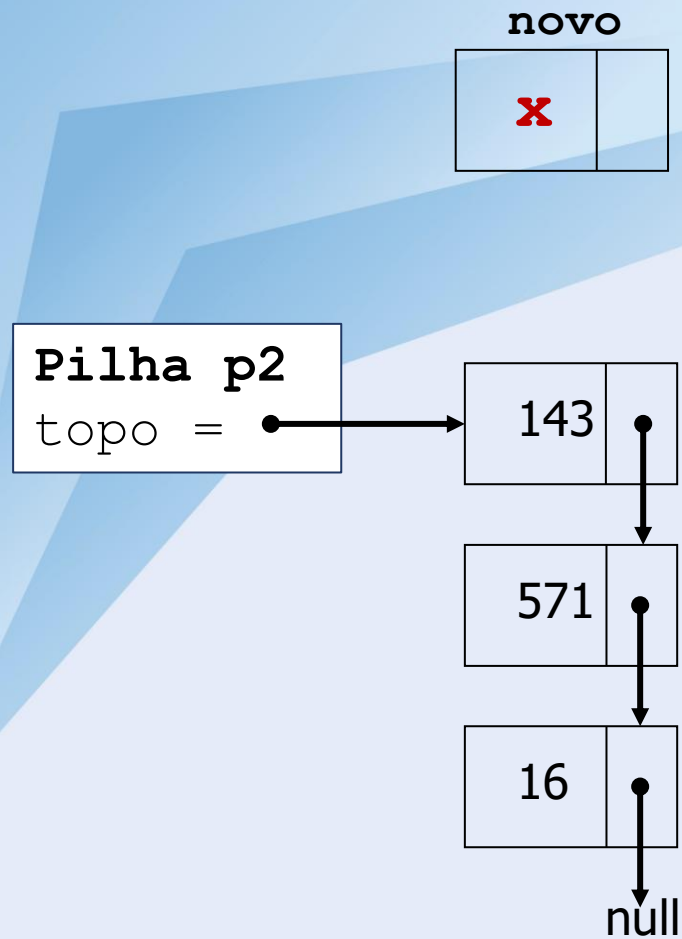


```
18     public void inserir(Object x){  
19         No novo = new No();  
20         novo.setValor(x);  
21         novo.setProx(this.topo);  
22         this.topo = novo;  
23     }
```

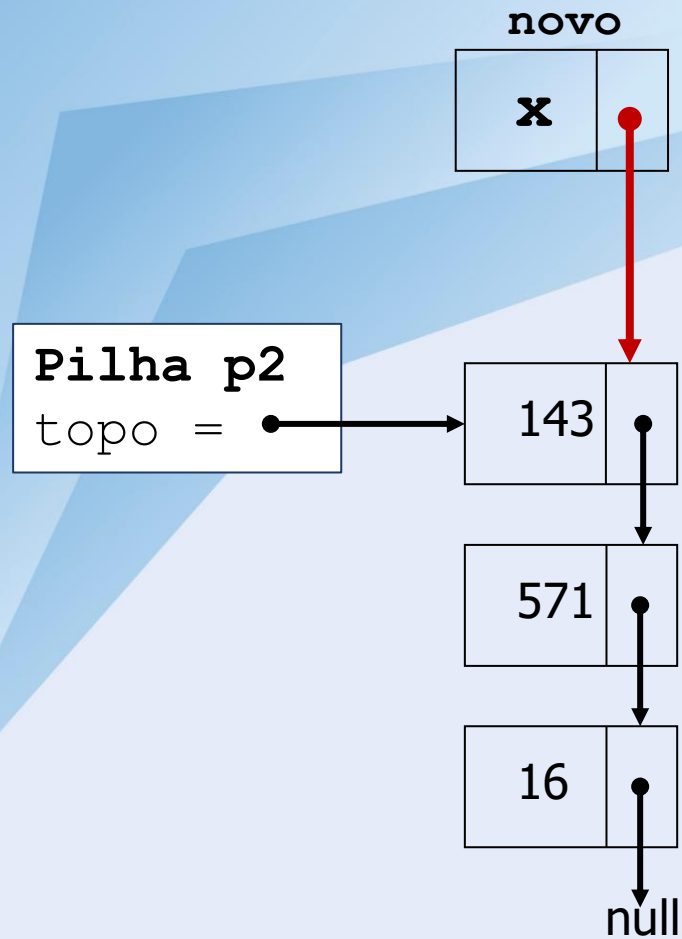
novο



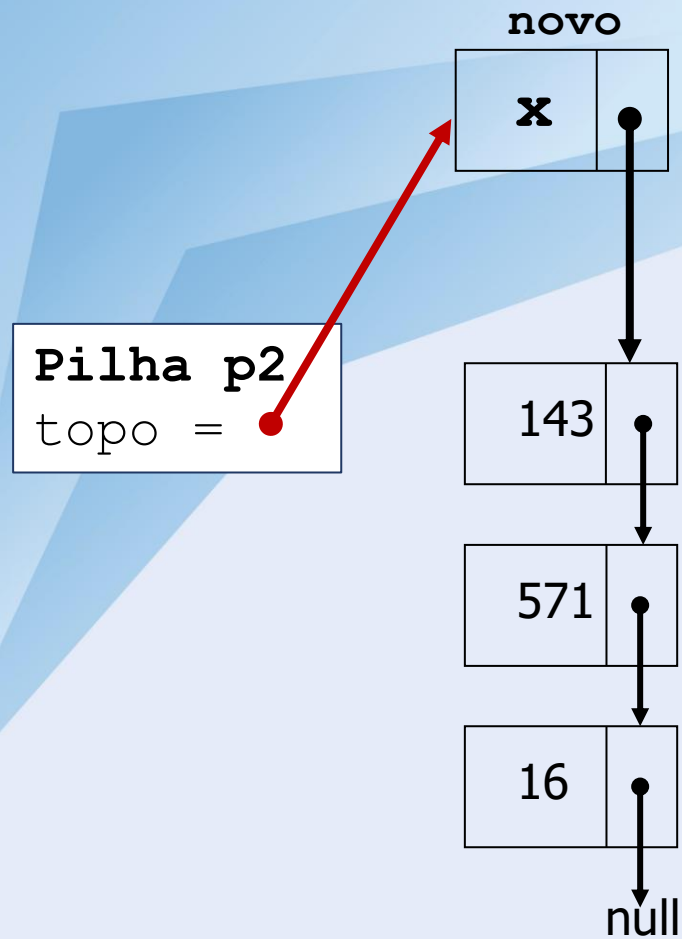
```
18     public void inserir(Object x){  
19         ➔ No novo = new No();  
20         novo.setValor(x);  
21         novo.setProx(this.topo);  
22         this.topo = novo;  
23     }
```



```
18     public void inserir(Object x){  
19         No novo = new No();  
20         → novo.setValor(x);  
21         novo.setProx(this.topo);  
22         this.topo = novo;  
23     }
```



```
18     public void inserir(Object x){
19         No novo = new No();
20         novo.setValor(x);
21         → novo.setProx(this.topo);
22         this.topo = novo;
23     }
```



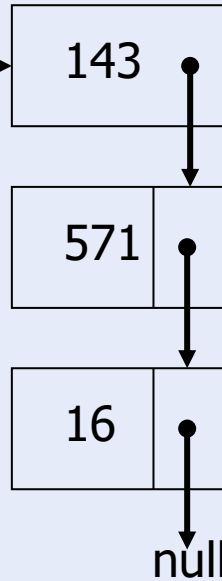
```
18     public void inserir(Object x){
19         No novo = new No();
20         novo.setValor(x);
21         novo.setProx(this.topo);
22         this.topo = novo;
23     }
```

- A operação **remove** (), remove o item que está no topo da pilha.

```
24     public void remover(){
25         if(this.vazia()){
26             System.out.println("Erro: Pilha vazia!");
27         }
28     }else{
29         System.out.println("Valor removido: "+this.topo.getValor());
30         this.topo = this.topo.getProx();
31     }
32 }
```

- Se a pilha estiver vazia, não tem nó para ser removido. Caso contrário, o topo é removido, e no novo topo da pilha será o próximo elemento da estrutura.

Pilha p2
topo = ●

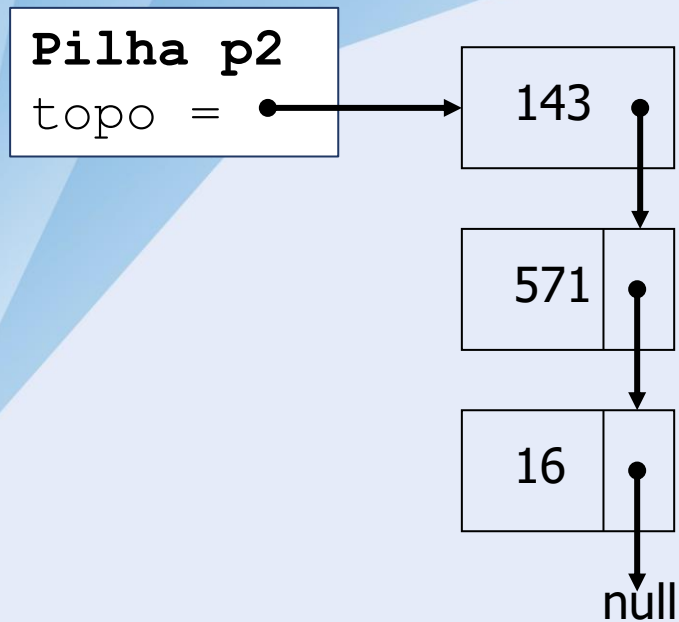


F



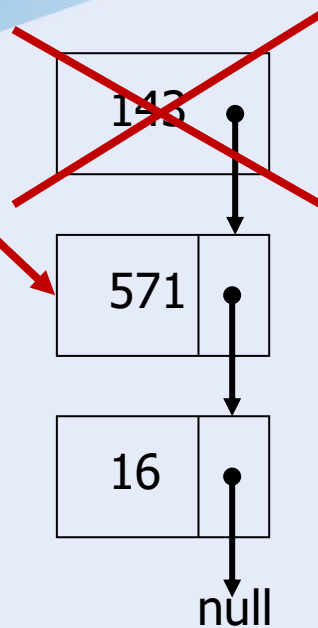
```
public void remover(){  
    if(this.vazia()){  
        System.out.println("Erro: Pilha vazia!");  
    }else{  
        System.out.println("Valor removido: "+this.topo.getValor());  
        this.topo = this.topo.getProx();  
    }  
}
```

Valor removido: 143



```
public void remover(){  
    if(this.vazia()){  
        System.out.println("Erro: Pilha vazia!");  
    }else{  
        System.out.println("Valor removido: "+this.topo.getValor());  
        this.topo = this.topo.getProx();  
    }  
}
```

Pilha p2
topo = ●



```
public void remover(){  
    if(this.vazia()){  
        System.out.println("Erro: Pilha vazia!");  
    }else{  
        System.out.println("Valor removido: "+this.topo.getValor());  
        → this.topo = this.topo.getProx();  
    }  
}
```

- A operação **mostrar()** apresenta todos os valores armazenados na pilha, começando pelo topo.

```
33     public void mostrar() {  
34         System.out.println("Pilha: ");  
35         for (No x = this.topo; x != null; x = x.getProx()) {  
36             System.out.print(x.getValor() + " -> ");  
37         }  
38         System.out.println("null");  
39     }
```

```
2  > public class Main {  
3  >     public static void main(String[] args) {  
4      Pilha p1 = new Pilha(null);  
5      p1.inserir(1);  
6      p1.inserir(2);  
7      p1.inserir(3);  
8      p1.inserir(4);  
9      p1.mostrar();  
10     p1.remover();  
11     p1.mostrar();  
12     }  
13 }
```

Run Main x



C:\Users\mnasu\.jdk\openjdk-24.0.1\bin\java.exe "-javaage

Pilha:

4 -> 3 -> 2 -> 1 -> null

Valor removido: 4

Pilha:

3 -> 2 -> 1 -> null

Process finished with exit code 0